

HEINRICH-HEINE-UNIVERSITÄT DÜSSELDORF

Bachelor Thesis

Implementierung und Evaluation der Reinforcement-Learning-Methoden Self-Play und Liga-Training in der Micro-RTS-Umgebung

Name!!

Degree:	B.Sc. Computer Science
Supervisor:	Professor Dr. Paul Swoboda
Co-Supervisor:	Dr. Peter Arndt
Faculty:	Mathematisch-Naturwissenschaftliche Fakultät

XXXXXXXXXXXX

Die vorliegende Arbeit verbessert einen bestehenden Agenten für das Echtzeit-Strategiespiel (RTS) MicroRTS. RTS-Spiele gelten als anspruchsvolle Domäne für Deep Reinforcement Learning, da Agenten mit großen kombinatorischen Aktionsräumen, simultanen Entscheidungen sowie verzögerten und oftmals spärlichen Belohnungen umgehen müssen. Als Basismethode verwenden wir Proximal Policy Optimization (PPO). Bei vielen Reinforcement-Learning (RL) -Trainingsarten, wie bei reinem PPO, kommt man an einen Punkt, an dem der Agent nicht effizient ausschließlich mit PPO weiter trainiert werden kann, weil ausreichend starke Gegner, gegen die man trainieren kann fehlen oder, weil die vorhandenen Gegner nicht divers genug sind, um eine robuste Policy zu entwickeln. Eine naheliegende und einfache Gegenmaßnahme ist Self-Play (SP), das jedoch in der Praxis instabil sein kann und zu Überanpassung oder zyklischen Strategien führt, wodurch die Leistung gegen externe Gegner sinken kann.

League-Training (LT) stabilisiert SP, indem es eine Population aus Agenten unterschiedlicher Trainingsstände und Rollen aufbaut. Der trainierenden Agent wird gezielt gegen vielfältige Gegner antreten. Unter anderem auch gegen Main-Exploiter, die trainiert wurden, Schwachstellen der aktuellen Policy auszunutzen. Wir evaluieren die resultierenden Agenten in Matches gegen verschiedene Ggnerstypen und vergleichen LT mit reinem SP sowie unserem Anfangs-Agenten. Der Anfangs-Agent wurde mit einer Mischung aus Behavior Cloning (BC) und PPO Trainiert. Unsere Ergebnisse zeigen, dass LT Robustheit und Generalisierungsfähigkeit des Agenten gegenüber unterschiedlichen Gegnern erhöht.

Contents

1. Introduction	2
2. Related Work	2
3. Method	3
3.1. Environment Setup	3
3.2. Neural Network Architecture	4
3.3. Behavior Cloning	4
3.4. Reinforcement Training	4
4. Experiments	4
4.1. Setup	4
4.2. Results	4
4.3. Discussion	4
5. Conclusion	4

1. Introduction

RL etablierte sich in den vergangenen Jahren als leistungsstarker Ansatz für die Entwicklung von Agenten in komplexen Umgebungen. Spielumgebungen wie RTS-Spiele stellen dabei eine besondere Herausforderung dar, mit großen Aktionsräumen, langfristiger Planung und simultanen Entscheidungen. Allerdings bieten sie klar definierte Zustandsräume, Belohnungsstrukturen und wiederholbare Experimente, was sie zu idealen und herausfordernden Testumgebungen für die Erforschung von RL-Methoden macht.

Micro-RTS ist eine vereinfachte Version von RTS-Spielen, die speziell für die Forschung entwickelt wurde [Huang et al., 2021]. Sie ist diskret und reduziert die visuelle und mechanische Komplexität von großen kommerziellen RTS-Spielen, zum Beispiel StarCraft II. Währenddessen behält sie die Kernschwierigkeiten, wie Ressourcenmanagement, Einheitenproduktion, Kampfmechaniken und taktische Positionierung, bei. Micro-RTS eignet sich deshalb für die systematische Untersuchung von Lernverfahren. Micro-RTS enthält weiterhin zahlreiche Trainingsherausforderungen, wie hohe kombinatorische Aktionsräume, verzögerte Belohnungen, Multi-Agent-Interaktionen und es erfordert auch robuster Strategien gegen unterschiedliche Gegner. Beispielsweise muss sich der Agent früh entscheiden, ob er gegen den aktuellen Gegner aggressiv oder defensiv spielen will, was sich auf die gesamte Spielstrategie auswirkt und eine langfristige Planung erfordert.

Ein zentrales Problem des RL ist die Generalisierung gegenüber unterschiedlichen Gegnern und Strategien: Agenten lernen oft Strategien, die stark auf den Trainingsgegner oder eine spezifische Spielsituation zugeschnitten sind. Das führt zu Überanpassung und eingeschränkter Robustheit. SP, Fictitious Self-Play (FSP) und LT sind Ansätze, die dieses Problem adressieren. In SP trainiert der Agent, in dem er gegen sich selbst spielt, was zu einem kontinuierlichen Lernprozess führt, der keine zusätzlichen diversen oder starken Gegner erfordert. Allerdings ist oft das Training instabil, weil es zyklisch lernt. Ein Beispiel: Spielt der Agentzustand B gut gegen A, C gut gegen B und A gut gegen C, könnte der Agent in einem Zyklus gefangen sein, in dem er immer nur den Agentenzustand wieder erlernt, der gegen sich selbst gut ist, aber nicht gegen andere Gegner. Um die Instabilitäten von reinem SP zu adressieren, wird in FSP nicht gegen den aktuellen Agenten sondern gegen ältere Versionen des Agenten gespielt, was ein zyklisches Lernen verhindert und in zwei Spieler Nullsummenspielen gegen das Nash-Gleichgewicht konvergiert. [Heinrich et al., 2015]. LT erweitert FSP, indem der Agent gegen historische Gegner aus einer Liga trainiert, die aus verschiedenen Agententypen besteht, beispielsweise aus historischen

Trainingsständen oder spezialisierten Strategien [Vinyals et al., 2019]. Ziel ist es, die Lernumgebung zu stabilisieren, in der Agenten gegen ein breites Spektrum an Strategien antreten und dadurch robuster werden.

Trotz der theoretischen Vorteile ist die praktische Implementierung von LT in komplexen Umgebungen, wie Micro-RTS, mit erheblichen Herausforderungen verbunden. Dazu gehören:

- die effiziente Verwaltung und Auswahl von Gegnern für das Training des Main-Agenten und für die spezialisierten Agenten, wie Main-Exploiter,
- die Stabilisierung des Trainingsprozesses bei nicht-konstanten Gegnerverteilungen,
- der Umgang mit großen Aktionsräumen und spärlichen oder verzögerten Belohnungen und
- die Bewertung der tatsächlichen Generalisierungsfähigkeit der trainierten Agenten.

Darüberhinaus werden wir die für die unterschiedlichen Trainingsparadigmen die Lernkurve, die Strategiediversität und die Robustheit vergleichen und evaluieren.

Der in diesem Projekt eingesetzte Basis-Agent, wird in einem dreistufigen Verfahren trainiert, das BC und BC-Fine-Tuning (BC-FT) mit anschließendem Fine-Tuning durch PPO kombiniert: Die beiden BC-Phasen ermöglichen dem Agenten, durch Imitation von Expertenverhalten schnell eine kompetente Basisstrategie zu erlernen [Torabi et al., 2018]. Dieser Ansatz des Trainings reduziert die anfängliche, oft ineffiziente Exploration in RL [Torabi et al., 2018]. Das Lernen von Grund auf mit RL-Methoden, wie PPO kann sehr zeitintensiv sein [Martin and Jhala, 2021]. PPO ist eine bekannte on-policy Policy-Optimierungsmethode, die die Policy-Update-Größe über eine Clipped-Surrogat-Zielfunktion begrenzt [Schulman et al., 2017]. PPO verbessert weiter die Leistung des BC-Agenten durch Exploration und Anpassung an die Umgebung, wodurch sich der Agent über die Expertenstrategien hinaus anpassen und generalisieren kann [Martin and Jhala, 2021]. Dieser zweistufige Ansatz, bei dem eine mit BC vor-trainierte Policy durch einen RL-Algorithmus verfeinert wird, hat sich als effektiv erwiesen, um die Trainingszeit zu verkürzen und die Gesamtleistung im Vergleich zu einem reinen RL-Ansatz zu verbessern [Martin and Jhala, 2021].

2. Related Work

Game Playing Agents MicroRTS Die dieser Arbeit zugrunde liegende (unveröffentlichte) Bachelorarbeit, implementierte verschiedene Agenten für Micro-RTS und kombinierte und evaluierte BC und PPO in der Micro-RTS

Umgebung [Huft, 2025]. Aus dieser Bachelorarbeit stammt der Basis-Agent und der initiale Code, der in dieser Arbeit weiter verbessert. Die dabei verwendete hybride Trainingspipeline besteht aus drei aufeinanderfolgenden Trainingsphasen: BC, BC-FT und abschließendem Training mittels PPO. [Huft, 2025].

Table 1

Bot	BC	BC-FT	PPO
WorkerRushAI	3	50	99
PassiveAI	98	100	100
LightRushAI	22	93	99
CoacAI	0	8	96
Mayari	0	5	95
RandomAI	52	95	99
RandomBiasedAI	18	68	86
rojo	0	34	84
mixedBot	4	44	59
izanagi	0	62	61
tiamat	11	78	84
droplet	0	31	59
guidedRojoA3N	4	42	75
naiveMCTS	0	5	3

MicroRTS-Py (Gym-MicroRTS)

RAISocketAI

BeClone

AlphaStar

MicroRTS-Py (Gym- μ RTS) Huang et al. stellen mit *Gym- μ RTS* bzw. *MicroRTS-Py* eine effiziente OpenAI-Gym-kompatible Schnittstelle für MicroRTS bereit und etablieren damit eine weit verbreitete DRL-Baseline [?]. In ihrer Arbeit zeigen sie, dass PPO in MicroRTS insbesondere dann konkurrenzfähig trainierbar ist, wenn geeignete Engineering-Maßnahmen eingesetzt werden, darunter *invalid action masking*, geeignete Aktions-/Beobachtungsrepräsentationen (z. B. GridNet) sowie architektonische Entscheidungen (u. a. IMPALA-CNN) [?, ?, ?]. Diese Arbeiten bilden den methodischen Ausgangspunkt vieler nachfolgender Deep-RL-Ansätze in MicroRTS.

RAISocketAI RAISocketAI ist der erste Deep-RL-Agent, der die IEEE MicroRTS Competition gewinnen konnte, und setzt damit einen wichtigen Referenzpunkt für DRL-basierte Ansätze in dieser Domäne [Goodfriend, 2024]. Zentral für die Performance sind laut Goodfriend ein iteratives Fine-Tuning der Policy, ein abgestimmtes Trainingsregime sowie Transfer-Learning zwischen Karten-/Problemgrößen [Goodfriend, 2024]. Zusätzlich beschreibt die Arbeit Erweiterungen und praktische

Verbesserungen an der MicroRTS-Py-Toolchain, die u. a. die Datengenerierung über Replays erleichtern und damit auch für Imitation-Learning-Ansätze relevant sind.

BeClone BeClone untersucht eine zweiphasige Trainingsstrategie für RTS-Agenten: Zunächst wird eine initiale Policy per BC aus Demonstrationen gelernt, anschließend erfolgt ein RL-Finetuning mittels *Advantage Actor-Critic* (A2C) im Self-Play [Martin and Jhala, 2021]. Der Fokus der Experimente liegt auf ressourcenorientierten Aufgaben und vergleichsweise kleinen Karten, wodurch BeClone primär als Nachweis dient, dass BC als Warm-Start die anschließende RL-Optimierung in RTS-Settings beschleunigen und stabilisieren kann [Martin and Jhala, 2021]. Im Gegensatz dazu zielt die vorliegende Arbeit auf die Implementierung und Evaluation von Self-Play und Liga-Training in einem stärker kompetitiven Setup mit PPO als Optimierungsverfahren.

AlphaStar AlphaStar demonstriert, dass *Liga-Training* (Population-Based Self-Play) in komplexen RTS-Domänen entscheidend zur Robustheit beitragen kann [Vinyals et al., 2019]. Das System wird zunächst durch Imitation Learning aus menschlichen Replays vortrainiert und anschließend in einer Liga aus unterschiedlichen Agententypen (u. a. Hauptagenten, Exploiters, historische Snapshots) weitertrainiert, um Überanpassung und Vergessen zu reduzieren und strategische Diversität zu fördern [Vinyals et al., 2019]. Obwohl MicroRTS und StarCraft II stark in Komplexität und Repräsentation differieren, liefert AlphaStar ein etabliertes Referenzdesign dafür, wie Self-Play durch ligaartige Matchmaking-Mechanismen zu stabileren und generalisierbaren Policies skaliert werden kann.

3. Method

3.1. Environment Setup

- T
- N
- H
- W
- C
-
-
-
-
-

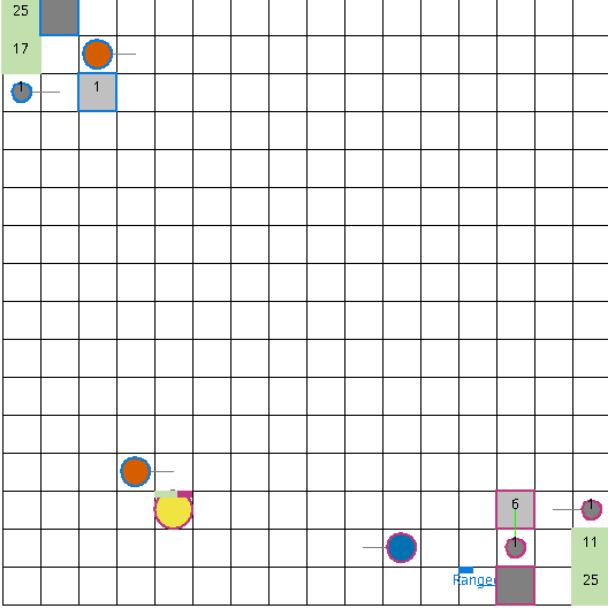


Figure 1. Screenshot basesWorkers16x16A map.

3.2. Neural Network Architecture

ScalarEncoder

$$\text{ScalarEncoder} : \mathbb{R}^{d_{in}} \rightarrow \mathbb{R}^{d_{out}},$$

3.3. Behavior Cloning

3.4. Reinforcement Training

4. Experiments

4.1. Setup

Table 2. Training parameters for PPO training.

Parameter	Value
Initial learning rate (Adam)	2.5×10^{-5}
Rollout steps per environment	512
Minibatch Size	3072
Max episode length	2,000 timesteps
Discount factor γ	1
Value function coefficient β	0.01
Entropy coefficient η	0.005
Gradient norm threshold ω	0.5
KL regularization coefficient λ_{KL}	0.4
GAE parameter λ	0.95

4.2. Results

4.3. Discussion

- **Generalization to random opponents:**

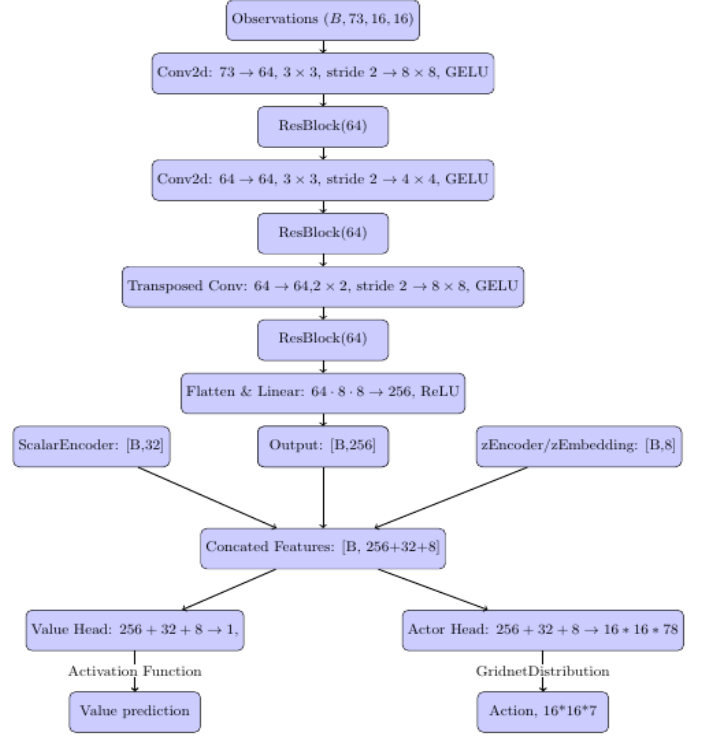


Figure 2. The Network Architecture

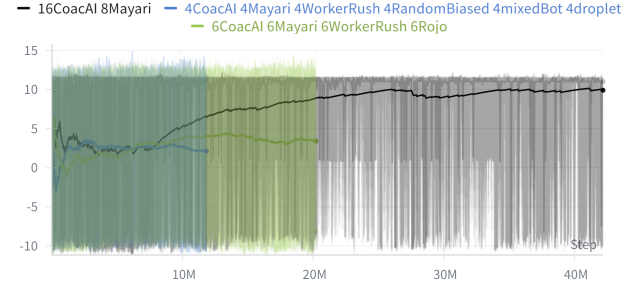


Figure 3. Episode rewards (y-axis) as a function of training timesteps (x-axis) on the BC-FT agent.

- **Map and observability constraints:**
- **Replay and training bottlenecks:**
- **Exploitability:**

5. Conclusion

References

- [Goodfriend, 2024] Goodfriend, S. (2024). Raisocketai: The first deep reinforcement learning agent to win the ieee microrts competition. In *2024 IEEE Conference on Games (CoG)*, pages 1–8. [3](#)

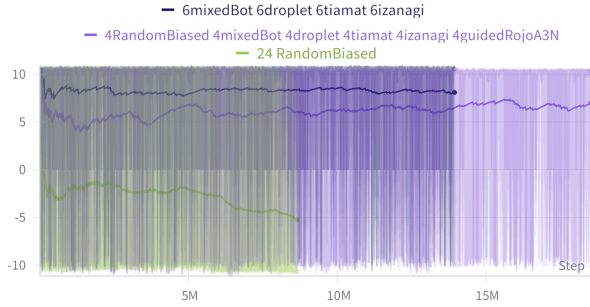


Figure 4. Episode rewards (y-axis) as a function of training timesteps (x-axis) on the final agent.

- [Heinrich et al., 2015] Heinrich, J., Lanctot, M., and Silver, D. (2015). Fictitious self-play in extensive-form games. In Bach, F. and Blei, D., editors, *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 805–813. 2
- [Huang et al., 2021] Huang, S., Ontanón, S., Bamford, C., and Grela, L. (2021). Gym-microrts: Toward affordable full game real-time strategy games research with deep reinforcement learning. *CoRR*, abs/2105.13807. 2
- [Huft, 2025] Huft, M. (2025). Game playing agents in microrts. 3
- [Martin and Jhala, 2021] Martin, D. and Jhala, A. (2021). Beclone: Behavior cloning with inference for real-time strategy games. In *Joint Proceedings of the AIIDE 2021 Workshops*. 2, 3
- [Schulman et al., 2017] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. *CoRR*, abs/1707.06347. 2
- [Torabi et al., 2018] Torabi, F., Warnell, G., and Stone, P. (2018). Behavioral cloning from observation. *arXiv preprint arXiv:1805.01954*. 2
- [Vinyals et al., 2019] Vinyals, O., Babuschkin, I., Czarnecki, W. M., Mathieu, M., Dudzik, A., Chung, J., Choi, D. H., Powell, R., Ewalds, T., Georgiev, P., Oh, J., Horgan, D., Kroiss, M., Danihelka, I., Huang, A., Sifre, L., Cai, T., Agapiou, J. P., Jaderberg, M., Vezhnevets, A. S., Leblond, R., Pohlen, T., Dalibard, V., Budden, D., Sulsky, Y., Molloy, J., Paine, T. L., Gulcehre, C., Wang, Z., Pfaff, T., Wu, Y., Ring, R., Yogatama, D., Wünsch, D., McKinney, K., Smith, O., Schaul, T., Lillicrap, T., Kavukcuoglu, K., Hassabis, D., Apps, C., and Silver, D. (2019). Grandmaster level in starcraft ii using multi-agent reinforcement learning. *Nature*, 575(7782):350–354. 2, 3

Erklärung

Hiermit versichere ich, dass ich die Bachelorarbeit selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Düsseldorf, den XXXXXXXXXXXX

Name!!